Introduction aux bases de données

Informatique — Licence 1 (L1)

Cours INF-L1-13

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Comprendre pourquoi les bases de données ont remplacé les fichiers classiques, modéliser des données avec Merise (MCD puis MLD), puis interroger une vraie base avec les requêtes SQL SELECT et un SGBD comme SQLite.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Données, information et limites des fichiers classiques
• Leçon 2 — Le modèle relationnel : tables, clés et relations
• Leçon 3 — Modélisation conceptuelle : entités et associations avec Merise
• Leçon 4 — Du schéma conceptuel au schéma relationnel
• Leçon 5 — Introduction au langage SQL : les requêtes SELECT de base
• Leçon 6 — Créer sa première base avec un SGBD comme SQLite
• Leçon 7 — Projet : base de données de gestion d'une bibliothèque
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Chaque jour, sans le savoir, tu utilises des dizaines de bases de données : ton compte WhatsApp retrouve instantanément tes contacts, la scolarité de l'IAM affiche ton emploi du temps, et la bibliothèque sait quel livre tu as emprunté et quand tu dois le rendre. Derrière chacune de ces réponses rapides se cache la même idée : ranger les données une seule fois, de façon structurée, pour pouvoir les retrouver, les croiser et les mettre à jour sans erreur. Ce cours t'apprend comment on conçoit un tel rangement, et comment on l'interroge.

Pendant longtemps, les organisations géraient leurs données dans de simples fichiers : un fichier pour les étudiants, un autre pour les paiements, un autre pour les notes. Cette approche finit toujours par créer des problèmes : la même adresse est recopiée dix fois, deux fichiers se contredisent, et répondre à une question simple demande des heures de recherche manuelle. Les bases de données relationnelles, inventées dans les années 1970, ont été créées précisément pour éliminer ces défauts grâce à une organisation en tables liées entre elles.

Dans ce cours, nous suivrons un fil rouge unique : la bibliothèque de l'IAM et ses emprunts de livres. Nous partirons du besoin réel, nous construirons le modèle conceptuel avec la méthode Merise, nous le transformerons en tables, puis nous créerons la base dans SQLite et nous répondrons en SQL à de vraies questions : quels livres sont encore dehors ? Qui est en retard ? Quel est le livre le plus emprunté ? À la fin, tu sauras reproduire cette démarche complète sur n'importe quel sujet de gestion.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Distinguer une donnée, une information et une base de données, et expliquer les limites des fichiers classiques (redondance, incohérences, accès difficiles).
- Définir les notions du modèle relationnel : relation (table), attribut, enregistrement, domaine, clé primaire et clé étrangère.
- Construire un Modèle Conceptuel de Données (MCD) avec la méthode Merise : entités, associations, propriétés et cardinalités.
- Appliquer les règles de passage du MCD au Modèle Logique de Données (MLD) : associations 1,N et N,N, placement des clés étrangères.
- Écrire des requêtes SQL SELECT de base : projection, sélection avec WHERE, opérateurs AND/OR/LIKE, tri avec ORDER BY et comptage avec COUNT.
- Créer une base complète avec un SGBD comme SQLite : CREATE TABLE, INSERT INTO et premières interrogations.
- Mener un mini-projet de bout en bout : analyser un besoin, modéliser, créer la base et répondre à des questions métier en SQL.

Leçon 1 — Données, information et limites des fichiers classiques

Une donnée est un fait brut, enregistré tel quel : « 14 », « Diarra », « 2026-09-05 ». Prise isolément, elle ne dit pas grand-chose. Une information, c'est une donnée placée dans un contexte qui lui donne du sens : « Awa Diarra a obtenu 14 en algorithmique » ou « le livre numéro 10 a été emprunté le 5 septembre 2026 ». Toute la gestion informatique consiste à stocker des données bien organisées pour pouvoir produire, à la demande, les informations dont on a besoin : un bulletin, une liste de retardataires, un état des stocks.

Une base de données est un ensemble structuré de données liées entre elles, géré par un logiciel spécialisé appelé SGBD (Système de Gestion de Base de Données) : MySQL, PostgreSQL, Oracle ou SQLite, que nous utiliserons dans ce cours. Le SGBD garantit que les données sont enregistrées une seule fois, que plusieurs personnes peuvent les consulter en même temps sans se gêner, et que des règles de cohérence sont respectées : impossible, par exemple, d'enregistrer un emprunt pour un adhérent qui n'existe pas.

Avant les SGBD, chaque application gérait ses propres fichiers. La bibliothèque aurait eu un fichier des adhérents, un fichier des prêts tenu par le bibliothécaire, et le secrétariat son propre fichier d'étudiants. Très vite apparaissent les trois maladies des fichiers classiques : la redondance (le nom et l'adresse d'un étudiant sont recopiés dans chaque fichier), les incohérences (l'adresse est corrigée dans un fichier mais pas dans les autres, on ne sait plus laquelle est vraie) et la rigidité (pour répondre à une nouvelle question, il faut souvent écrire un nouveau programme qui relit tout). C'est pour guérir ces trois maladies qu'existent les bases de données.

L'essentiel
Une donnée est un fait brut ; une information est une donnée interprétée dans son contexte. Une base de données centralise les données dans un SGBD pour éviter redondances, incohérences et difficultés d'accès des fichiers classiques.

Exemple — Le même étudiant recopié dans trois fichiers : la redondance en action

```
Fichier INSCRIPTIONS.txt (scolarité)
E101 ; Diarra ; Awa ; L1 Informatique ; awa@iam.ml
E102 ; Traoré ; Moussa ; L2 BBA ; moussa@iam.ml

Fichier EMPRUNTS.txt (bibliothèque, tenu à part)
Diarra Awa ; Algorithmique pas à pas ; 2026-09-05
Diarra Awa ; Bases de données pour débutants ; 2026-09-20

Fichier COTISATIONS.txt (association des étudiants)
Diarra ; Awa ; ancienne adresse : rue 12, Bamako

Problèmes visibles :
- « Diarra Awa » est recopié 4 fois (redondance).
- L'adresse de la cotisation n'est plus à jour (incohérence).
- Qui a emprunté quoi ? Il faut relire tout le fichier à la main.
```

À retenir

- Donnée = fait brut enregistré ; information = donnée mise en contexte pour répondre à une question.
- Le SGBD est le logiciel qui stocke, protège et interroge la base de données.
- Les fichiers classiques souffrent de redondance, d'incohérences et d'accès rigides aux données.

Exercice

La cantine de l'IAM tient un fichier papier des repas : pour chaque ticket vendu, on y écrit le nom de l'étudiant, sa classe, le plat choisi et le prix. Le gérant veut savoir combien d'étudiants de L1 Informatique ont mangé un thiéboudienne ce mois-ci. Explique pourquoi cette question est pénible avec le fichier papier, et ce qu'une base de données changerait.

✔ Corrigé
Avec le fichier papier, il faut relire tous les tickets du mois un par un, repérer ceux où la classe vaut « L1 Informatique » ET le plat vaut « thiéboudienne », puis compter à la main : c'est long et source d'erreurs. De plus, le nom de l'étudiant est recopié sur chaque ticket (redondance) et une faute d'orthographe sur un nom peut fausser le comptage (incohérence). Avec une base de données, l'étudiant est enregistré une seule fois dans une table ETUDIANT, chaque ticket devient une ligne d'une table VENTE reliée à l'étudiant, et la question devient une simple requête de sélection avec comptage, répondue en une fraction de seconde, sans recopie ni relecture manuelle.

Leçon 2 — Le modèle relationnel : tables, clés et relations

Le modèle relationnel, proposé par Edgar Codd en 1970, organise les données en tables (on dit aussi relations). Une table ressemble à un tableau : les colonnes sont les attributs (par exemple nom, prenom, filiere pour un adhérent), chaque ligne est un enregistrement qui décrit un exemplaire concret (un adhérent précis), et l'ensemble des valeurs possibles d'un attribut s'appelle son domaine (pour l'attribut niveau, le domaine est L1, L2, L3). Toutes les lignes d'une table ont la même structure, et l'ordre des lignes n'a aucune importance.

Pour retrouver une ligne sans ambiguïté, chaque table possède une clé primaire : un attribut (ou un groupe d'attributs) dont la valeur est unique pour chaque ligne et jamais vide. À la bibliothèque, chaque adhérent reçoit un numéro d'inscription unique, idAdherent : deux étudiantes peuvent s'appeler Awa Diarra, mais elles n'auront jamais le même numéro. Choisir une clé primaire artificielle (un numéro attribué par le système) plutôt qu'une donnée naturelle comme le nom évite presque tous les problèmes de doublons.

Les tables ne vivent pas isolées : elles sont reliées par les clés étrangères. Une clé étrangère est un attribut d'une table qui contient la clé primaire d'une autre table, créant ainsi un lien. Dans la table EMPRUNT, l'attribut idAdherent est une clé étrangère : il désigne l'adhérent qui a fait l'emprunt, sans recopier son nom ni son adresse. C'est ce mécanisme qui supprime la redondance : l'information sur l'adhérent n'existe qu'à un seul endroit, dans la table ADHERENT, et tous les emprunts y font référence par le numéro.

L'essentiel
Une table = des attributs en colonnes et des enregistrements en lignes. La clé primaire identifie chaque ligne de façon unique ; la clé étrangère relie une table à une autre en référençant sa clé primaire, sans recopier les données.

Exemple — La table LIVRE de la bibliothèque et sa clé primaire

```
Table LIVRE (clé primaire : idLivre)

idLivre | titre | auteur | categorie | annee | nbExemplaires
--------+------------------------------+------------+--------------+-------+--------
------
10 | Algorithmique pas à pas | C. Delannoy| Informatique | 2019 | 3
11 | Droit des affaires OHADA | A. Sow | Droit | 2021 | 2
12 | Marketing : principes et | P. Kotler | Gestion | 2018 | 4
| pratiques | | | |
13 | Bases de données pour débutants| J. Dupont | Informatique | 2022 | 2
14 | Comptabilité générale | M. Koné | Gestion | 2020 | 5

- 6 attributs (colonnes), 5 enregistrements (lignes).
- idLivre identifie chaque livre sans ambiguïté : c'est la clé primaire.
- Le domaine de « categorie » est ici : Informatique, Droit, Gestion.
```

À retenir

- Relation = table ; attribut = colonne ; enregistrement (ou n-uplet) = ligne.
- La clé primaire est unique et jamais vide : elle identifie chaque ligne.
- La clé étrangère crée le lien entre tables en référençant une clé primaire, sans duplication.

Exercice

Un commerçant veut une table PRODUIT pour sa boutique avec les attributs suivants : nom du produit, prix, quantité en stock, nom du fournisseur, téléphone du fournisseur. a) Ce schéma contient-il un problème de conception ? Lequel ? b) Propose une clé primaire pour PRODUIT. c) Comment corriger le schéma pour éviter la redondance ?

✔ Corrigé
a) Oui : le nom et le téléphone du fournisseur seront recopiés sur chaque produit fourni par le même fournisseur. Si le fournisseur change de numéro, il faudra corriger toutes les lignes, avec un risque d'oubli et donc d'incohérence : c'est exactement la maladie des fichiers classiques, recréée dans une table. b) On ajoute un attribut artificiel idProduit (numéro unique attribué par le système) comme clé primaire : le nom du produit ne peut pas servir de clé, car deux produits différents peuvent porter le même nom. c) On crée une deuxième table FOURNISSEUR(idFournisseur, nom, telephone) et la table PRODUIT ne conserve qu'une clé étrangère idFournisseur qui désigne le fournisseur. L'information du fournisseur n'existe alors qu'en un seul exemplaire.

Leçon 3 — Modélisation conceptuelle : entités et associations avec Merise

Avant de créer des tables, il faut comprendre le besoin : c'est le rôle du Modèle Conceptuel de Données (MCD), première étape de la méthode Merise, très utilisée dans les pays francophones. Le MCD décrit les données du problème sans se préoccuper de l'informatique : on y repère les entités, c'est-à-dire les objets importants que l'on veut gérer (un adhérent, un livre, un étudiant, un cours), et pour chaque entité ses propriétés, les informations qui la décrivent (le livre a un titre, un auteur, une année). Chaque entité possède un identifiant, la propriété qui distingue chaque exemplaire des autres.

Les entités sont reliées par des associations, qui expriment les liens du monde réel : un adhérent EMPRUNTE un livre, un étudiant SUIT un cours, un enseignant DISPENSE un cours. Une association peut porter ses propres propriétés : l'emprunt a une date d'emprunt et une date de retour prévue, qui n'appartiennent ni à l'adhérent seul, ni au livre seul, mais bien au lien entre les deux. Repérer ces propriétés d'association est un point clé de l'analyse.

Chaque association est précisée par ses cardinalités, qui indiquent combien d'exemplaires peuvent participer de chaque côté. On les lit des deux côtés : un adhérent peut emprunter de 0 à plusieurs livres (cardinalité 0,N côté livre), et un livre peut être emprunté par 0 à plusieurs adhérents au fil du temps (0,N côté adhérent). Les valeurs possibles sont 0 ou 1 pour le minimum, 1 ou N pour le maximum. Les

cardinalités ne se devinent pas : elles se déduisent des règles de gestion énoncées par l'utilisateur, par exemple « un étudiant ne peut pas emprunter plus de 3 livres en même temps » préciserait un maximum, même si dans notre modèle simplifié nous garderons N.

L'essentiel
Le MCD de Merise décrit le besoin en entités (avec identifiant et propriétés), associations (parfois porteuses de propriétés) et cardinalités (0,1 / 1,1 / 0,N / 1,N) déduites des règles de gestion.

Exemple — Le MCD de la bibliothèque décrit en texte

```
ENTITÉ ADHERENT
idAdherent (identifiant)
nom, prenom, filiere, niveau, email

ENTITÉ LIVRE
idLivre (identifiant)
titre, auteur, categorie, annee, nbExemplaires

ASSOCIATION EMPRUNTER
ADHERENT ---- (0,N) ---- EMPRUNTER ---- (0,N) ---- LIVRE
Propriétés de l'association :
dateEmprunt, dateRetourPrevue, dateRetourEffective

Lecture des cardinalités :
- Un adhérent peut emprunter de 0 à plusieurs livres.
- Un livre peut être emprunté par 0 à plusieurs adhérents
(au fil du temps, pas au même moment).
```

À retenir

- Entité = objet géré (adhérent, livre) ; chaque entité a un identifiant unique.
- Association = lien entre entités ; elle peut porter ses propres propriétés (dates d'emprunt).
- Les cardinalités (min, max) de chaque côté traduisent les règles de gestion du réel.

Exercice

Pour la scolarité de l'IAM, on veut gérer des étudiants et des cours (matières). Un étudiant suit plusieurs cours, un cours est suivi par plusieurs étudiants, et pour chaque étudiant dans chaque cours on connaît sa note finale. a) Quelles sont les entités et leurs identifiants ? b) Quelle est l'association, ses propriétés et ses cardinalités ?

✔ Corrigé
a) Entité ETUDIANT, identifiant idEtudiant (numéro d'inscription), propriétés nom, prenom, filiere, niveau. Entité COURS, identifiant idCours (code de la matière, par exemple INF-L1-13), propriétés intitule, coefficient, semestre. b) L'association SUIVRE relie ETUDIANT et COURS. La note finale est une propriété de l'association : elle ne dépend ni de l'étudiant seul, ni du cours seul, mais du couple (étudiant, cours). Cardinalités : un étudiant suit de 1 à plusieurs cours (1,N — il est inscrit à au moins un cours) ; un cours est suivi par 0 à plusieurs étudiants (0,N — un cours peut exister avant toute inscription). C'est donc une association de type N,N, comme EMPRUNTER dans le MCD de la bibliothèque.

Leçon 4 — Du schéma conceptuel au schéma relationnel

Le MCD décrit le besoin, mais un SGBD ne connaît que des tables : il faut donc transformer le modèle conceptuel en Modèle Logique de Données (MLD), c'est-à-dire en schéma relationnel. La transformation suit des règles précises, qui dépendent des cardinalités des associations. Première règle, toujours vraie : chaque entité devient une table, ses propriétés deviennent les attributs de la table, et son identifiant devient la clé primaire.

Deuxième cas, le plus fréquent : l'association de type 1,N (on dit aussi « un à plusieurs »). Si chaque livre est rangé dans un seul rayon et qu'un rayon contient plusieurs livres, on ne crée pas de table pour l'association : on ajoute simplement, dans la table LIVRE (du côté N), une clé étrangère idRayon qui désigne le rayon du livre. La règle se retient ainsi : la clé étrangère migre du côté où la cardinalité maximale vaut N... plus précisément, elle se place dans la table correspondant à l'entité dont la cardinalité maximale, de son côté de l'association, est N, et elle référence l'autre entité. Créer une table séparée serait inutile et compliquerait toutes les requêtes.

Troisième cas : l'association de type N,N (cardinalités maximales N des deux côtés), comme EMPRUNTER entre ADHERENT et LIVRE ou SUIVRE entre ETUDIANT et COURS. Ici, l'association devient elle-même une table. Cette table reçoit comme clés étrangères les identifiants des deux entités liées, plus les propriétés de l'association (les dates, la note). Pour notre table EMPRUNT, nous ajouterons en plus un identifiant propre, idEmprunt, comme clé primaire : il simplifie la désignation d'un emprunt précis (par exemple pour enregistrer son retour). À défaut, la clé primaire serait composée du couple (idAdherent, idLivre) complété par la date d'emprunt, car un même adhérent peut emprunter le même livre plusieurs fois dans l'année.

L'essentiel
Règles de passage MCD → MLD : chaque entité devient une table ; une association 1,N se traduit par une clé étrangère placée du côté N ; une association N,N devient une table qui contient les deux clés étrangères et les propriétés de l'association.

Exemple — Le MLD de la bibliothèque (clés primaires soulignées, # = clé étrangère)

```
ADHERENT (idAdherent, nom, prenom, filiere, niveau, email)
----------
LIVRE (idLivre, titre, auteur, categorie, annee,
------- nbExemplaires, #idRayon)
RAYON (idRayon, nomRayon, etage)
-------
EMPRUNT (idEmprunt, #idAdherent, #idLivre, dateEmprunt,
--------- dateRetourPrevue, dateRetourEffective)

D'où vient chaque élément ?
- ADHERENT, LIVRE : les deux entités du MCD.
- RAYON et #idRayon dans LIVRE : l'association RANGER est
de type 1,N (un livre est dans UN rayon, un rayon contient
PLUSIEURS livres) -> clé étrangère du côté N, dans LIVRE.
- EMPRUNT : l'association EMPRUNTER est de type N,N, elle
devient une table avec les deux clés étrangères et les
dates, propriétés de l'association.
```

À retenir

- Entité → table ; identifiant → clé primaire ; propriétés → attributs.
- Association 1,N → pas de nouvelle table : une clé étrangère du côté N.
- Association N,N → une nouvelle table, avec les clés des deux entités et les propriétés de l'association.

Exercice

Dans une entreprise, un employé travaille dans un seul département, et un département regroupe plusieurs employés. Par ailleurs, un employé peut participer à plusieurs projets, et un projet mobilise plusieurs employés ; pour chaque participation, on connaît le nombre d'heures consacrées. Donne le MLD complet (tables, clés primaires et clés étrangères) en appliquant les règles de passage.

✔ Corrigé
Entités : EMPLOYE (idEmploye, nom, fonction) et DEPARTEMENT (idDepartement, nomDepartement) deviennent deux tables. L'association TRAVAILLER est de type 1,N (un employé dans un seul département, plusieurs employés par département) : on ajoute donc la clé étrangère #idDepartement dans la table EMPLOYE, sans créer de table pour l'association. L'association PARTICIPER est de type N,N et porte la propriété nbHeures : elle devient une table PARTICIPATION(#idEmploye, #idProjet, nbHeures), dont la clé primaire est le couple (idEmploye, idProjet) — un employé participe une fois à un projet donné — et PROJET (idProjet, nomProjet, budget) est la table issue de l'entité PROJET. MLD final : DEPARTEMENT(idDepartement, nomDepartement) ; EMPLOYE(idEmploye, nom, fonction, #idDepartement) ; PROJET(idProjet, nomProjet, budget) ; PARTICIPATION(#idEmploye, #idProjet, nbHeures).

Leçon 5 — Introduction au langage SQL : les requêtes SELECT de base

SQL (Structured Query Language) est le langage standard pour dialoguer avec les SGBD relationnels : créer des tables, y insérer des données et surtout les interroger. La requête d'interrogation fondamentale est le SELECT. Sa forme minimale est : SELECT liste_des_attributs FROM nom_de_la_table ; Elle affiche les colonnes demandées pour toutes les lignes de la table. L'étoile, SELECT *, demande toutes les colonnes. On appelle projection le choix des colonnes, et c'est la première moitié de presque toutes les questions que l'on pose à une base.

La deuxième moitié est la sélection des lignes, faite avec la clause WHERE suivie d'une condition : WHERE categorie = 'Informatique' ne garde que les livres de cette catégorie. Les conditions se combinent avec AND (les deux conditions vraies), OR (au moins une) et NOT, et utilisent les comparaisons =, <>, <, >, <=, >=. Deux détails importants : les valeurs texte s'écrivent entre apostrophes, et pour tester l'absence de valeur on n'écrit jamais = NULL mais IS NULL — par exemple, un emprunt dont la dateRetourEffective IS NULL est un emprunt encore en cours, le livre n'est pas revenu. L'opérateur LIKE permet enfin les recherches approchées sur les textes : le motif '%données%' signifie « contient le mot données », le signe % remplaçant n'importe quelle suite de caractères.

Pour présenter les résultats, ORDER BY trie les lignes : ORDER BY annee DESC donne les livres du plus récent au plus ancien (DESC = décroissant, ASC = croissant, choix par défaut). La fonction COUNT(*) compte les lignes qui satisfont la condition : combinée à WHERE, elle répond aux questions « combien ? » sans avoir à afficher toutes les lignes. Enfin, quand une question porte sur deux tables liées, on les réunit avec JOIN ... ON, qui recrée les liens grâce aux clés : c'est ainsi que l'on obtient, pour chaque emprunt en cours, le nom de l'adhérent et le titre du livre alors que la table EMPRUNT ne contient que des numéros.

L'essentiel
SELECT choisit les colonnes, FROM la table, WHERE filtre les lignes (AND, OR, LIKE, IS NULL), ORDER BY trie, COUNT(*) compte, et JOIN relie les tables par leurs clés pour répondre aux questions qui croisent plusieurs tables.

Exemple — Quatre questions, quatre requêtes — sur les données de la bibliothèque

```
-- Q1. Titres et auteurs des livres d'informatique
SELECT titre, auteur FROM LIVRE
WHERE categorie = 'Informatique';
-- Résultat : 2 lignes
-- Algorithmique pas à pas | C. Delannoy
-- Bases de données pour débutants| J. Dupont

-- Q2. Adhérents de L1 Informatique
SELECT nom, prenom FROM ADHERENT
WHERE filiere = 'Informatique' AND niveau = 'L1';
-- Résultat : Diarra Awa ; Sangaré Mariam

-- Q3. Les livres du plus récent au plus ancien
SELECT titre, annee FROM LIVRE ORDER BY annee DESC;
-- Résultat : 2022, 2021, 2020, 2019, 2018 (5 lignes)

-- Q4. Combien d'emprunts sont encore en cours ?
SELECT COUNT(*) FROM EMPRUNT
WHERE dateRetourEffective IS NULL;
-- Résultat : 4
```

À retenir

- SELECT = colonnes, FROM = table, WHERE = condition sur les lignes, dans cet ordre.
- Une valeur absente se teste avec IS NULL, jamais avec = NULL.
- LIKE et % permettent les recherches « contient » ; ORDER BY ... DESC trie en ordre décroissant.

Exercice

Sur les tables de la bibliothèque (voir les données complètes à la leçon 7) : a) Écris la requête qui affiche le titre et le nombre d'exemplaires des livres de la catégorie Gestion. b) Écris la requête qui affiche les emprunts (idEmprunt, idLivre, dateEmprunt) réalisés par l'adhérent numéro 1. c) Que retourne SELECT titre FROM LIVRE WHERE titre LIKE '%données%' ; ?

✔ Corrigé

a) SELECT titre, nbExemplaires FROM LIVRE WHERE categorie = 'Gestion' ; Elle retourne deux lignes : « Marketing : principes et pratiques » (4 exemplaires) et « Comptabilité générale » (5 exemplaires). b) SELECT idEmprunt, idLivre, dateEmprunt FROM EMPRUNT WHERE idAdherent = 1 ; Elle retourne les emprunts 1 (livre 10, le 2026-09-05) et 2 (livre 13, le 2026-09-20). c) Le motif '%données%' signifie « le titre contient le mot données » : une seule ligne correspond, « Bases de données pour débutants ». Attention aux accents : LIKE compare le texte tel qu'il est enregistré, le motif doit donc contenir l'accent de « données ».

Leçon 6 — Créer sa première base avec un SGBD comme SQLite

SQLite est un SGBD relationnel complet mais très léger : toute la base tient dans un seul fichier, il n'y a aucun serveur à installer, et il est intégré à de nombreux outils gratuits (le programme en ligne de commande sqlite3, ou l'interface graphique DB Browser for SQLite). C'est le choix idéal pour apprendre et pour de petites applications ; les gros systèmes utiliseront plutôt MySQL ou PostgreSQL, mais le SQL que tu écris ici restera le même à quelques détails près.

Créer une table se fait avec l'ordre CREATE TABLE, qui déclare le nom de la table, ses attributs et leur type, et les contraintes : PRIMARY KEY pour la clé primaire, NOT NULL pour interdire l'absence de valeur, et REFERENCES pour déclarer une clé étrangère. Les types de base de SQLite sont simples : INTEGER pour les entiers, TEXT pour les textes (les dates, au format AAAA-MM-JJ, y sont rangées comme textes, ce qui permet de les comparer dans l'ordre chronologique), REAL pour les nombres décimaux. Une fois les tables créées, on les remplit avec INSERT INTO ... VALUES, qui ajoute une ligne à chaque exécution — ou plusieurs lignes d'un coup en séparant les groupes de valeurs par des virgules.

La démarche complète ressemble à un petit chantier : on crée le fichier de base, on exécute les CREATE TABLE dans le bon ordre (une table contenant une clé étrangère se crée après la table qu'elle référence, donc ADHERENT et LIVRE avant EMPRUNT), on insère les données, puis on vérifie immédiatement avec quelques SELECT que tout est en place. Si une erreur de frappe survient, le SGBD la signale et refuse l'ordre fautif : c'est la garantie que la base ne contient que des données conformes au schéma. On peut alors interroger la base librement, comme à la leçon précédente.

L'essentiel
Avec SQLite, la base est un simple fichier. CREATE TABLE déclare la structure (types, PRIMARY KEY, NOT NULL, REFERENCES), INSERT INTO ajoute les lignes, et SELECT vérifie le résultat : on crée d'abord les tables référencées, puis celles qui contiennent les clés étrangères.

Exemple — Création de la base bibliotheque.db et premiers enregistrements

```
-- 1. Les tables (ADHERENT et LIVRE avant EMPRUNT)
CREATE TABLE ADHERENT (
idAdherent INTEGER PRIMARY KEY,
nom TEXT NOT NULL, prenom TEXT NOT NULL,
filiere TEXT, niveau TEXT, email TEXT
);
CREATE TABLE LIVRE (
idLivre INTEGER PRIMARY KEY,
titre TEXT NOT NULL, auteur TEXT,
categorie TEXT, annee INTEGER, nbExemplaires INTEGER
);
CREATE TABLE EMPRUNT (
idEmprunt INTEGER PRIMARY KEY,
idAdherent INTEGER NOT NULL REFERENCES ADHERENT(idAdherent),
idLivre INTEGER NOT NULL REFERENCES LIVRE(idLivre),
dateEmprunt TEXT NOT NULL,
dateRetourPrevue TEXT,
dateRetourEffective TEXT
);

-- 2. Quelques données
INSERT INTO ADHERENT VALUES
(1, 'Diarra', 'Awa', 'Informatique', 'L1', 'awa.diarra@iam.ml'),
(2, 'Traoré', 'Moussa', 'BBA', 'L2', 'moussa.traore@iam.ml');
INSERT INTO LIVRE VALUES
(10, 'Algorithmique pas à pas', 'C. Delannoy', 'Informatique', 2019, 3),
(13, 'Bases de données pour débutants', 'J. Dupont', 'Informatique', 2022, 2);
INSERT INTO EMPRUNT VALUES
(2, 1, 13, '2026-09-20', '2026-10-04', NULL);

-- 3. Vérification
SELECT titre FROM LIVRE WHERE categorie = 'Informatique';
```

À retenir

• SQLite = un SGBD complet dans un seul fichier, parfait pour apprendre et prototyper.

• CREATE TABLE fixe types et contraintes ; PRIMARY KEY et REFERENCES protègent la cohérence.

• On crée les tables dans l'ordre des dépendances : les tables référencées avant celles qui référencent.

Exercice

a) Écris l'ordre CREATE TABLE d'une table RAYON(idRayon, nomRayon, etage) avec clé primaire. b) La table LIVRE du MLD possède une clé étrangère #idRayon : écris la ligne à ajouter dans le CREATE TABLE de LIVRE pour la déclarer. c) Un étudiant exécute d'abord le CREATE TABLE de EMPRUNT, avant ceux de ADHERENT et LIVRE. Est-ce un problème avec la contrainte REFERENCES telle qu'écrite dans l'exemple du cours ? Explique.

✔ Corrigé
a) CREATE TABLE RAYON (idRayon INTEGER PRIMARY KEY, nomRayon TEXT NOT NULL, etage INTEGER) ; b) Il faut d'abord la colonne idRayon INTEGER parmi les attributs de LIVRE, puis sa déclaration en clé étrangère : soit en ligne, idRayon INTEGER REFERENCES RAYON(idRayon), soit en fin de table par la contrainte FOREIGN KEY (idRayon) REFERENCES RAYON(idRayon). c) Avec SQLite, le CREATE TABLE de EMPRUNT réussit même si les tables référencées n'existent pas encore : SQLite ne vérifie la cible de la clé étrangère qu'au moment des insertions (et seulement si les clés étrangères sont activées par PRAGMA foreign_keys = ON). En revanche, les INSERT dans EMPRUNT échoueront tant que les adhérents et livres correspondants n'existent pas. La bonne habitude reste donc : créer les tables référencées d'abord, insérer ensuite dans le même ordre — la plupart des autres SGBD refusent d'ailleurs de créer une clé étrangère vers une table absente.

Leçon 7 — Projet : base de données de gestion d'une bibliothèque

Ce projet rassemble tout le cours en suivant la démarche complète d'un informaticien : analyser le besoin, modéliser (MCD), transformer (MLD), créer la base (SQLite) et l'interroger (SQL). Le besoin est celui de notre fil rouge : la bibliothécaire de l'IAM veut enregistrer les adhérents et les livres, suivre les emprunts et les retours, repérer les retards et connaître les livres les plus demandés. Nous avons déjà le MCD (leçon 3) et le MLD (leçon 4) ; dans le projet, la table RAYON est laissée de côté pour rester concentré sur les emprunts, et il ne reste qu'à créer les trois tables ADHERENT, LIVRE et EMPRUNT et à les remplir du jeu de données complet présenté dans l'exemple ci-dessous.

Une fois la base créée, les questions métier deviennent des requêtes. La liste des emprunts en cours exige un JOIN : la table EMPRUNT ne contient que des numéros, il faut la relier à ADHERENT pour le nom et à LIVRE pour le titre. Les retards se repèrent en combinant deux conditions : le livre n'est pas revenu (dateRetourEffective IS NULL) et la date de retour prévue est dépassée (dateRetourPrevue < date du jour). Enfin, le livre le plus emprunté s'obtient en regroupant les emprunts par livre avec GROUP BY, en comptant chaque groupe avec COUNT(*) et en triant par ce compte : c'est un premier pas vers les requêtes d'analyse, que tu approfondiras en L2.

Vérifions les résultats sur notre jeu de données, à la date du 11 octobre 2026 : quatre emprunts sont en cours (numéros 2, 4, 5 et 6) ; parmi eux, deux sont en retard — l'emprunt 2 d'Awa Diarra (retour prévu le 4 octobre) et l'emprunt 4 d'Ibrahim Coulibaly (prévu le 9 octobre). Le livre le plus emprunté est « Algorithmique pas à pas » avec 3 emprunts, devant tous les autres livres qui n'en comptent qu'un. Ces résultats, que l'on peut contrôler à la main sur le petit jeu de données, sont exactement ceux que les requêtes de l'exemple retournent : prends toujours le temps de vérifier une requête sur un exemple connu avant de lui faire confiance sur de grandes données.

L'essentiel
Le projet enchaîne la démarche complète : besoin → MCD → MLD → création SQLite → requêtes métier. Les questions réelles combinent JOIN (retrouver noms et titres), conditions sur les dates (retards) et GROUP BY avec COUNT (statistiques).

Exemple — Le jeu de données complet et les trois requêtes métier du projet

```
EMPRUNT (idEmprunt, idAdherent, idLivre, dates : emprunt / retour prévu / retour
effectif)
1 | adh 1 | livre 10 | 2026-09-05 / 2026-09-19 / 2026-09-18
2 | adh 1 | livre 13 | 2026-09-20 / 2026-10-04 / (en cours)
3 | adh 2 | livre 12 | 2026-09-22 / 2026-10-06 / 2026-10-05
4 | adh 4 | livre 10 | 2026-09-25 / 2026-10-09 / (en cours)
5 | adh 5 | livre 10 | 2026-10-01 / 2026-10-15 / (en cours)
6 | adh 3 | livre 11 | 2026-10-02 / 2026-10-16 / (en cours)
ADHERENT : 1 Diarra Awa (Info L1), 2 Traoré Moussa (BBA L2),
3 Keïta Fatoumata (Droit L1), 4 Coulibaly Ibrahim (Info L3),
5 Sangaré Mariam (Info L1)

-- Emprunts en cours, avec noms et titres
SELECT A.nom, A.prenom, L.titre, E.dateRetourPrevue
FROM EMPRUNT E
JOIN ADHERENT A ON A.idAdherent = E.idAdherent
JOIN LIVRE L ON L.idLivre = E.idLivre
WHERE E.dateRetourEffective IS NULL;
-- 4 lignes : Diarra (livre 13), Coulibaly (livre 10),
-- Sangaré (livre 10), Keïta (livre 11)

-- Retards au 11 octobre 2026
SELECT idEmprunt, idAdherent FROM EMPRUNT
WHERE dateRetourEffective IS NULL
AND dateRetourPrevue < '2026-10-11';
-- 2 lignes : emprunts 2 et 4

-- Livre le plus emprunté
SELECT idLivre, COUNT(*) AS nb FROM EMPRUNT
GROUP BY idLivre ORDER BY nb DESC;
-- livre 10 : 3 ; livres 11, 12, 13 : 1 chacun
```

À retenir

- Une démarche de projet réussie enchaîne toujours : analyse, MCD, MLD, création, interrogation.
- Le JOIN est indispensable dès que la réponse mélange des informations de plusieurs tables.
- On valide chaque requête sur un petit jeu de données dont on connaît la réponse à la main.

Exercice

La bibliothécaire veut deux nouvelles réponses, à la date du 11 octobre 2026 : a) le nombre d'emprunts réalisés par les adhérents de la filière Informatique ; b) la liste des

titres des livres qui n'ont jamais été empruntés. Écris les requêtes SQL correspondantes et donne leurs résultats sur le jeu de données du projet. Pour la question b, tu peux utiliser la sous-requête : WHERE idLivre NOT IN (SELECT idLivre FROM EMPRUNT).

✔ Corrigé
a) Il faut relier EMPRUNT à ADHERENT pour connaître la filière de l'emprunteur : SELECT COUNT(*) FROM EMPRUNT E JOIN ADHERENT A ON A.idAdherent = E.idAdherent WHERE A.filiere = 'Informatique' ; Les adhérents d'informatique sont les numéros 1, 4 et 5 ; leurs emprunts sont les numéros 1, 2 (adhérent 1), 4 (adhérent 4) et 5 (adhérent 5) : le résultat est 4. b) SELECT titre FROM LIVRE WHERE idLivre NOT IN (SELECT idLivre FROM EMPRUNT) ; Les livres empruntés au moins une fois sont 10, 13, 12 et 11 ; il reste le livre 14, « Comptabilité générale », qui n'a jamais été emprunté : la requête retourne donc une seule ligne, ce titre. Remarque comme la réponse à la question b aurait été très pénible à obtenir en relisant des fichiers d'emprunts à la main.

Travaux dirigés (TD)

TD 1 — Modéliser la scolarité de l'IAM : du besoin au MLD

L'administration de l'IAM veut informatiser le suivi des cours. Règles de gestion : un étudiant est inscrit dans une seule classe (par exemple L1 Informatique, groupe A) ; une classe regroupe plusieurs étudiants. Un enseignant peut dispenser plusieurs cours, et un cours est dispensé par un seul enseignant. Un étudiant suit plusieurs cours, un cours est suivi par plusieurs étudiants, et l'on conserve la note finale de chaque étudiant dans chaque cours. Un cours a un code, un intitulé et un coefficient ; un enseignant a un nom et une spécialité ; un étudiant a un numéro d'inscription, un nom et un prénom ; une classe a un code et un effectif maximal. Travail demandé : 1) liste les entités, leurs identifiants et leurs propriétés ; 2) décris les associations avec leurs cardinalités et leurs éventuelles propriétés ; 3) donne le MLD complet (tables, clés primaires, clés étrangères) en justifiant chaque choix par la règle de passage utilisée.

✔ Corrigé du TD 1
1) Entités : ETUDIANT(idEtudiant, nom, prenom) ; CLASSE(idClasse, code — par exemple « L1-INFO-A » —, effectifMax) ; ENSEIGNANT(idEnseignant, nom, specialite) ; COURS(idCours, intitule, coefficient). 2) Associations : INSCRIPTION entre ETUDIANT et CLASSE, cardinalités (1,1) côté étudiant (chaque étudiant est dans une seule classe) et (1,N) côté classe ; DISPENSER entre ENSEIGNANT et COURS, cardinalités (0,N) côté enseignant (il peut dispenser plusieurs cours, éventuellement aucun en début d'année) et (1,1) côté cours (un cours a exactement un enseignant responsable) ; SUIVRE entre ETUDIANT et COURS, de type N,N, avec la propriété noteFinale. 3) MLD : CLASSE(idClasse, code, effectifMax). ENSEIGNANT(idEnseignant, nom, specialite). ETUDIANT(idEtudiant, nom, prenom, #idClasse) : INSCRIPTION est de type 1,N, la clé étrangère #idClasse est donc placée du côté N, dans ETUDIANT. COURS(idCours, intitule, coefficient, #idEnseignant) : DISPENSER est aussi de type 1,N, clé étrangère dans COURS, du côté du cours qui ne participe qu'une fois. SUIVRE(#idEtudiant, #idCours, noteFinale) : SUIVRE est de type N,N, elle devient une table contenant les deux clés étrangères ; sa clé primaire est le couple (#idEtudiant, #idCours), car un étudiant a une seule note finale par cours suivi.

TD 2 — Interroger la base de la bibliothèque

On reprend la base bibliotheque.db du projet (leçon 7) avec ses trois tables et son jeu de données complet : 5 adhérents, 5 livres, 6 emprunts. Écris les requêtes SQL qui répondent aux questions suivantes, et donne le résultat exact de chacune : 1) Quels sont les titres des livres parus à partir de 2020, triés du plus ancien au plus récent ? 2) Quels adhérents (nom, prénom) sont en L1, toutes filières confondues ? 3) Combien de livres la bibliothèque possède-t-elle en tout, toutes catégories confondues, si l'on additionne les exemplaires ? (Utilise la fonction SUM sur nbExemplaires.) 4) Pour chaque emprunt déjà retourné, affiche le numéro de l'emprunt et sa durée réelle en jours ne sont pas demandés ; affiche seulement idEmprunt, idLivre et dateRetourEffective des emprunts terminés, triés par date de retour croissante. 5) Quels sont les noms des adhérents qui ont au moins un emprunt en retard au 11 octobre 2026 ? (JOIN nécessaire.)

✔ Corrigé du TD 2
1) SELECT titre FROM LIVRE WHERE annee >= 2020 ORDER BY annee ASC ; Résultat : « Comptabilité générale » (2020), « Droit des affaires OHADA » (2021), « Bases de données pour débutants » (2022) — 3 lignes. 2) SELECT nom, prenom FROM ADHERENT WHERE niveau = 'L1' ; Résultat : Diarra Awa, Keïta Fatoumata, Sangaré Mariam — 3 lignes (les adhérents 1, 3 et 5). 3) SELECT SUM(nbExemplaires) FROM LIVRE ; Les exemplaires sont 3 + 2 + 4 + 2 + 5 = 16 : le résultat est 16. Attention à ne pas confondre avec COUNT(*), qui compterait les titres (5) et non les exemplaires. 4) SELECT idEmprunt, idLivre, dateRetourEffective FROM EMPRUNT WHERE dateRetourEffective IS NOT NULL ORDER BY dateRetourEffective ASC ; Résultat : emprunt 1 (livre 10, retour le 2026-09-18) puis emprunt 3 (livre 12, retour le 2026-10-05) — 2 lignes. 5) SELECT DISTINCT A.nom FROM ADHERENT A JOIN EMPRUNT E ON E.idAdherent = A.idAdherent WHERE E.dateRetourEffective IS NULL AND E.dateRetourPrevue < '2026-10-11' ; Les emprunts en retard sont les numéros 2 (adhérent 1, Diarra) et 4 (adhérent 4, Coulibaly) : le résultat est Diarra et Coulibaly. Le mot DISTINCT évite de répéter un nom si un adhérent avait plusieurs retards.

Travaux pratiques (TP)

TP 1 — Construire la base bibliotheque.db sous SQLite, de zéro

À l'aide de DB Browser for SQLite ou du programme sqlite3, réalise la base complète du projet. Étapes : 1) crée un nouveau fichier bibliotheque.db ; 2) exécute les trois CREATE TABLE de la leçon 6 (ADHERENT, LIVRE, EMPRUNT) ; 3) insère les 5 adhérents et les 5 livres du cours, puis les 6 emprunts du tableau de la leçon 7 (la date de retour effective d'un emprunt en cours sera insérée avec la valeur NULL) ; 4) vérifie le contenu de chaque table avec SELECT * ; 5) exécute les requêtes de la leçon 5 (Q1 à Q4) puis les trois requêtes métier de la leçon 7, et compare chaque résultat avec celui annoncé dans le cours ; 6) ajoute un nouvel adhérent de ton choix et un nouvel emprunt pour le livre 14, puis relance la requête « livres jamais empruntés » de l'exercice de la leçon 7 : que doit-elle retourner maintenant ?

✔ Attendus du TP 1
Résultats attendus : SELECT * FROM ADHERENT donne 5 lignes, LIVRE 5 lignes, EMPRUNT 6 lignes. Leçon 5 : Q1 retourne les 2 livres d'informatique (titres des livres 10 et 13) ; Q2 retourne Diarra Awa et Sangaré Mariam ; Q3 retourne les 5 livres dans l'ordre des années 2022, 2021, 2020, 2019, 2018 ; Q4 retourne 4 emprunts en cours. Leçon 7 : la liste JOIN des emprunts en cours donne 4 lignes (Diarra/livre 13, Coulibaly/livre 10, Sangaré/livre 10, Keïta/livre 11) ; les retards au 2026-10-11 sont les emprunts 2 et 4 ; le livre le plus emprunté est le livre 10 avec 3 emprunts. Étape 6 : après avoir emprunté le livre 14, la requête des livres jamais empruntés retourne zéro ligne — ce qui prouve qu'elle suit bien le contenu réel de la base et non une réponse figée. Si un de tes résultats diffère, cherche d'abord une erreur de saisie dans les INSERT (date inversée, numéro de livre faux) avant de remettre la requête en cause : dans neuf cas sur dix, c'est la donnée qui est fausse, pas la requête.

Exercices corrigés supplémentaires

Exercice 1

On te propose la table unique suivante pour gérer les emprunts : EMPRUNT_LARG(idEmprunt, nomAdherent, filiereAdherent, titreLivre, auteurLivre, dateEmprunt). Cite deux problèmes concrets de ce schéma (avec un scénario pour chacun) et propose le schéma corrigé en tables, avec les clés.

✔ Corrigé

Problème 1, redondance et incohérence : le nom et la filière de l'adhérent sont recopiés à chaque emprunt ; si Awa Diarra passe de L1 à L2, il faut corriger toutes ses lignes d'emprunt, et une ligne oubliée crée deux informations contradictoires. Problème 2, anomalie d'insertion et de suppression : impossible d'enregistrer un nouveau livre (titre, auteur) tant que personne ne l'a emprunté, puisque le titre n'existe que dans les lignes d'emprunt ; symétriquement, supprimer le dernier emprunt d'un livre fait disparaître le livre de la base. Schéma corrigé : ADHERENT(idAdherent, nom, filiere...) ; LIVRE(idLivre, titre, auteur...) ; EMPRUNT(idEmprunt, #idAdherent, #idLivre, dateEmprunt...) : chaque fait n'est enregistré qu'une fois, et les tables existent indépendamment les unes des autres.

Exercice 2

Chaque livre de la bibliothèque est rangé dans un seul rayon (par exemple « Informatique », au 1er étage), et un rayon contient plusieurs livres. a) Quelles sont les cardinalités de l'association RANGER entre LIVRE et RAYON, lues des deux côtés ? b) Lors du passage au MLD, faut-il créer une table RANGER ? Où place-t-on la clé étrangère, et pourquoi ? c) Donne le MLD des deux tables.

✔ Corrigé

a) Du côté du livre : un livre est rangé dans exactement un rayon, cardinalité (1,1). Du côté du rayon : un rayon contient de 0 à plusieurs livres, cardinalité (0,N) — 0 car un rayon nouvellement installé peut être encore vide. b) Non, on ne crée pas de table RANGER : l'association est de type 1,N. La clé étrangère #idRayon est placée dans la table LIVRE, du côté où la cardinalité maximale vaut N (plusieurs livres peuvent partager la même valeur de idRayon, chaque livre n'en a qu'une). Placer à l'inverse #idLivre dans RAYON obligerait à répéter le rayon pour chaque livre ou à ranger plusieurs numéros dans une seule case, ce qui détruirait la structure de table. c) RAYON(idRayon, nomRayon, etage) ; LIVRE(idLivre, titre, auteur, categorie, annee, nbExemplaires, #idRayon).

Exercice 3

Sur la base de la bibliothèque (données des leçons 5 et 7), écris les requêtes SQL suivantes et donne leur résultat : a) les titres des livres dont il reste au moins 4 exemplaires ; b) le nombre total d'emprunts enregistrés ; c) les adhérents (nom et filière) dont le nom commence par la lettre S (pense à LIKE 'S%').

✔ Corrigé
a) SELECT titre FROM LIVRE WHERE nbExemplaires >= 4 ; Résultat : « Marketing : principes et pratiques » (4 exemplaires) et « Comptabilité générale » (5 exemplaires) — 2 lignes. b) SELECT COUNT(*) FROM EMPRUNT ; Résultat : 6, le nombre de lignes de la table, retournés et en cours confondus (aucune condition WHERE). c) SELECT nom, filiere FROM ADHERENT WHERE nom LIKE 'S%' ; Le motif 'S%' signifie « commence par S, suivi de n'importe quoi » : une seule ligne correspond, Sangaré (filière Informatique). Attention, certains SGBD distinguent majuscules et minuscules dans LIKE, d'autres non ; ici le nom est enregistré avec un S majuscule, le résultat est le même dans les deux cas.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quelle est la différence entre une donnée et une information ?

   A) Il n'y en a aucune, ce sont des synonymes

   B) Une donnée est un fait brut ; une information est une donnée placée dans un contexte qui lui donne du sens

   C) Une information est toujours un nombre, une donnée est toujours un texte

2. Quel rôle joue un SGBD ?

   A) Il stocke, organise, protège et permet d'interroger les données de la base

   B) Il sert uniquement à dessiner les schémas MCD

   C) Il remplace le système d'exploitation de l'ordinateur

3. Dans le modèle relationnel, la clé primaire d'une table sert à...

   A) Trier les lignes par ordre alphabétique

   B) Stocker le mot de passe de la base

   C) Identifier chaque ligne de façon unique, sans doublon ni valeur vide

4. Une clé étrangère est...

   A) Une clé primaire venue d'un autre pays

   B) Un attribut qui référence la clé primaire d'une autre table pour créer un lien

   C) Une clé secondaire utilisée uniquement pour les recherches

5. Dans un MCD Merise, une association peut-elle porter des propriétés ?

   A) Non, seules les entités ont des propriétés

   B) Oui, uniquement si l'association est réflexive

   C) Oui : par exemple la date d'emprunt appartient à l'association EMPRUNTER, ni à l'adhérent seul, ni au livre seul

6. Lors du passage du MCD au MLD, une association de type 1,N devient...

   A) Une nouvelle table obligatoire

   B) Une clé étrangère placée dans la table du côté N

   C) Un attribut texte libre dans les deux tables

7. Lors du passage du MCD au MLD, une association de type N,N devient...

A) Une nouvelle table contenant les clés étrangères des deux entités et les propriétés de l'association

B) Une clé étrangère dans chacune des deux tables d'entités

C) Elle est supprimée, car impossible à représenter

8. Quelle requête affiche les titres des livres de la catégorie Droit ?

A) SELECT * WHERE categorie = 'Droit' FROM LIVRE

B) AFFICHER titre DE LIVRE SI categorie = 'Droit'

C) SELECT titre FROM LIVRE WHERE categorie = 'Droit'

9. Pour trouver les emprunts dont le livre n'est pas encore revenu, la bonne condition est...

A) WHERE dateRetourEffective = NULL

B) WHERE dateRetourEffective IS NULL

C) WHERE dateRetourEffective = 'en cours'

10. À quoi sert la clause ORDER BY dans une requête SELECT ?

A) À regrouper les lignes par catégorie

B) À limiter le nombre de lignes affichées à 10

C) À trier les lignes du résultat, par exemple ORDER BY annee DESC pour aller du plus récent au plus ancien

Corrigés du QCM

1. Réponse B — « 2026-10-04 » est une donnée ; « le retour du livre était prévu le 2026-10-04 » est une information.

2. Réponse A — Le SGBD (SQLite, MySQL, PostgreSQL...) est le logiciel gestionnaire de la base : stockage, cohérence, accès simultanés et interrogation en SQL.

3. Réponse C — La clé primaire (par exemple idAdherent) distingue chaque enregistrement ; deux lignes ne peuvent pas partager la même valeur.

4. Réponse B — Dans EMPRUNT, idAdherent est une clé étrangère : elle désigne l'adhérent sans recopier ses informations.

5. Réponse C — Les propriétés qui dépendent du lien entre deux entités (dates d'emprunt, note d'un étudiant dans un cours) appartiennent à l'association.

6. Réponse B — Exemple : RANGER (1,N) se traduit par #idRayon dans la table LIVRE, sans créer de table supplémentaire.

7. Réponse A — EMPRUNTER (N,N) devient la table EMPRUNT, avec #idAdherent, #idLivre et les dates, propriétés de l'association.

8. Réponse C — La syntaxe est SELECT colonnes FROM table WHERE condition ; les valeurs texte s'écrivent entre apostrophes.

9. Réponse B — NULL représente l'absence de valeur et se teste avec IS NULL (ou IS NOT NULL) ; la comparaison = NULL ne retourne jamais de ligne.

10. Réponse C — ORDER BY trie le résultat selon une ou plusieurs colonnes, en ordre croissant (ASC, par défaut) ou décroissant (DESC).

Résumé — les notions importantes

• Une donnée est un fait brut, une information est une donnée en contexte ; les bases de données existent pour centraliser les données et éviter redondances et incohérences des fichiers classiques.

• Le SGBD est le logiciel qui gère la base : il stocke les tables, fait respecter les contraintes et répond aux requêtes SQL.

• Modèle relationnel : table (relation), attributs, enregistrements ; la clé primaire identifie chaque ligne, la clé étrangère relie les tables sans recopier les données.

• MCD (Merise) : entités avec identifiant et propriétés, associations parfois porteuses de propriétés, cardinalités déduites des règles de gestion.

• Passage au MLD : entité → table ; association 1,N → clé étrangère du côté N ; association N,N → nouvelle table avec les deux clés et les propriétés de l'association.

• SQL SELECT : projection des colonnes, WHERE pour filtrer (AND, OR, LIKE, IS NULL), ORDER BY pour trier, COUNT(*) pour compter, JOIN pour croiser les tables.

• Démarche complète d'un projet de base de données : analyser le besoin, construire le MCD, le transformer en MLD, créer la base (par exemple sous SQLite) et vérifier chaque requête sur un jeu de données connu.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Données, information et limites des fichiers classiques
Donnée = fait brut enregistré ; information = donnée mise en contexte pour répondre à une question. ; Le SGBD est le logiciel qui stocke, protège et interroge la base de données. ; Les fichiers classiques souffrent de redondance, d'incohérences et d'accès rigides aux données.

Fiche 2 — Le modèle relationnel : tables, clés et relations
Relation = table ; attribut = colonne ; enregistrement (ou n-uplet) = ligne. ; La clé primaire est unique et jamais vide : elle identifie chaque ligne. ; La clé étrangère crée le lien entre tables en référençant une clé primaire, sans duplication.

Fiche 3 — Modélisation conceptuelle : entités et associations avec Merise
Entité = objet géré (adhérent, livre) ; chaque entité a un identifiant unique. ; Association = lien entre entités ; elle peut porter ses propres propriétés (dates d'emprunt). ; Les cardinalités (min, max) de chaque côté traduisent les règles de gestion du réel.

Fiche 4 — Du schéma conceptuel au schéma relationnel
Entité → table ; identifiant → clé primaire ; propriétés → attributs. ; Association 1,N → pas de nouvelle table : une clé étrangère du côté N. ; Association N,N → une nouvelle table, avec les clés des deux entités et les propriétés de l'association.

Fiche 5 — Introduction au langage SQL : les requêtes SELECT de base
SELECT = colonnes, FROM = table, WHERE = condition sur les lignes, dans cet ordre. ; Une valeur absente se teste avec IS NULL, jamais avec = NULL. ; LIKE et % permettent les recherches « contient » ; ORDER BY ... DESC trie en ordre décroissant.

Fiche 6 — Créer sa première base avec un SGBD comme SQLite
SQLite = un SGBD complet dans un seul fichier, parfait pour apprendre et prototyper. ; CREATE TABLE fixe types et contraintes ; PRIMARY KEY et REFERENCES protègent la cohérence. ; On crée les tables dans l'ordre des dépendances : les tables référencées avant celles qui référencent.

Fiche 7 — Projet : base de données de gestion d'une bibliothèque
Une démarche de projet réussie enchaîne toujours : analyse, MCD, MLD, création, interrogation. ; Le JOIN est indispensable dès que la réponse mélange des informations de plusieurs tables. ; On valide chaque requête sur un petit jeu de données dont on connaît la réponse à la main.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 2 — Cours Modèle Relationnel et MCD

Chaîne : E mds · Langue : Français · Niveau : Débutant à intermédiaire
Un cours complet qui présente d'abord les règles du modèle relationnel (relations, clés, dépendances fonctionnelles), puis le MCD de Merise avec les cardinalités. Il fait exactement le pont entre les leçons 2, 3 et 4 de ce cours.

https://www.youtube.com/watch?v=VFHVNA8xgK0

Leçon 3 — Créer un MCD [MODELE CONCEPTUEL DE DONNEES] - Exemple en Ligne Simple | Formation MySQL

Chaîne : Numelion IT · Langue : Français · Niveau : Débutant
La construction d'un MCD pas à pas avec l'outil en ligne Mocodo : repérer les entités, placer les associations, les attributs et les cardinalités sur un exemple concret. Parfait pour visualiser la leçon 3 avant de passer au MLD.

https://www.youtube.com/watch?v=BMwTb_fPqE4

Leçon 4 — Créer un MLD (Modèle Logique de Données) depuis un MCD - Méthode & Exemple - Bases de données

Chaîne : Fabinou · Langue : Français · Niveau : Débutant
La méthode de conversion d'un MCD en MLD expliquée avec un exemple : les entités deviennent des tables, les associations 1,N donnent des clés étrangères et les associations N,N deviennent des tables. Un résumé en images des règles de la leçon 4, avec une vérification sur tableur.

https://www.youtube.com/watch?v=Ak6eIfzipSQ

Leçon 5 — Base de données (Part.1): Initiation SQL & algèbre relationnelle

Chaîne : Schraf : Maths-info · Langue : Français · Niveau : Débutant

Une initiation au SQL qui met chaque requête en parallèle avec l'opération d'algèbre relationnelle correspondante, en s'exerçant directement dans SQLite en ligne. Idéal pour comprendre ce que fait réellement un SELECT avant d'écrire les tiens.

https://www.youtube.com/watch?v=nG65ew-3Oh0

Leçon 6 — Apprendre SQL pour Débutant - 01 - Créer une base de données

Chaîne : Ecole Numérique du Génie en Programmation (ENGP) · Langue : Français · Niveau : Débutant

Le premier épisode d'une série pour débutants : ce qu'est SQL, l'installation d'un SGBD (MySQL, SQLite ou PostgreSQL) et la création de ta première base de données, étape par étape. Le compagnon pratique de la leçon 6 et du TP.

https://www.youtube.com/watch?v=YWJ9aLbjJE8

Sources et références

• Gardarin, G. — Bases de données (Eyrolles) : l'ouvrage de référence francophone sur le modèle relationnel, la conception et le langage SQL.

• Hainaut, J.-L. — Bases de données : concepts, utilisation et développement (Dunod) : très complet sur la modélisation conceptuelle et la méthode Merise.

• Date, C. J. — Introduction aux bases de données (Vuibert) : le classique international sur le modèle relationnel, traduit en français.

• Documentation officielle de SQLite — sqlite.org : référence technique du SGBD utilisé dans les travaux pratiques de ce cours (en anglais).

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.